10 基本資料結構

Part III Data Structures

本章定位

基本資料結構的困難常在表示法與操作前提：物理陣列順序不等於queue順序，節點handle不等於鍵，free slot也不等於清零。本章以具體前後狀態證明queue、串列、配置器與樹表示的連結不變量和成本。

前置知識

陣列、指標、抽象資料型態與漸近分析。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 用陣列或鏈結表示堆疊與佇列。
- 維持 linked list 的操作不變量與邊界條件。
- 以索引鏈結表示沒有原生指標的物件。
- 10.1 Stacks and queues
- 10.2 Linked lists
- 10.3 Implementing pointers and objects
- 10.4 Representing rooted trees

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

10.1 Stacks and queues

ADT（abstract data type，抽象資料型態）先規定操作的語義，再選表示法。Stack的pop移除最後加入且尚未移除的元素，即LIFO；queue的dequeue移除最早加入且尚未移除的元素，即FIFO。兩者的delete都由規則決定對象，不接收任意鍵或任意位置。

固定容量m≥1的陣列stack，本章採0-index、top=有效元素數，資料佔S[0..top−1]，0≤top≤m。

```
push(x): if top==m: error overflow
         S[top]=x; top++
pop():   if top==0: error underflow
         top--; return S[top]
```

Push前先檢查，才不會寫越界；pop前先檢查，才不會減到−1。歸納看每次只延伸或縮短有效前綴，最後有效值正是最近push，故滿足LIFO。Pop後陣列可能還留舊bits，但不屬於邏輯stack；是否清除引用是記憶體管理的額外問題。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

環狀Queue的表示不變量

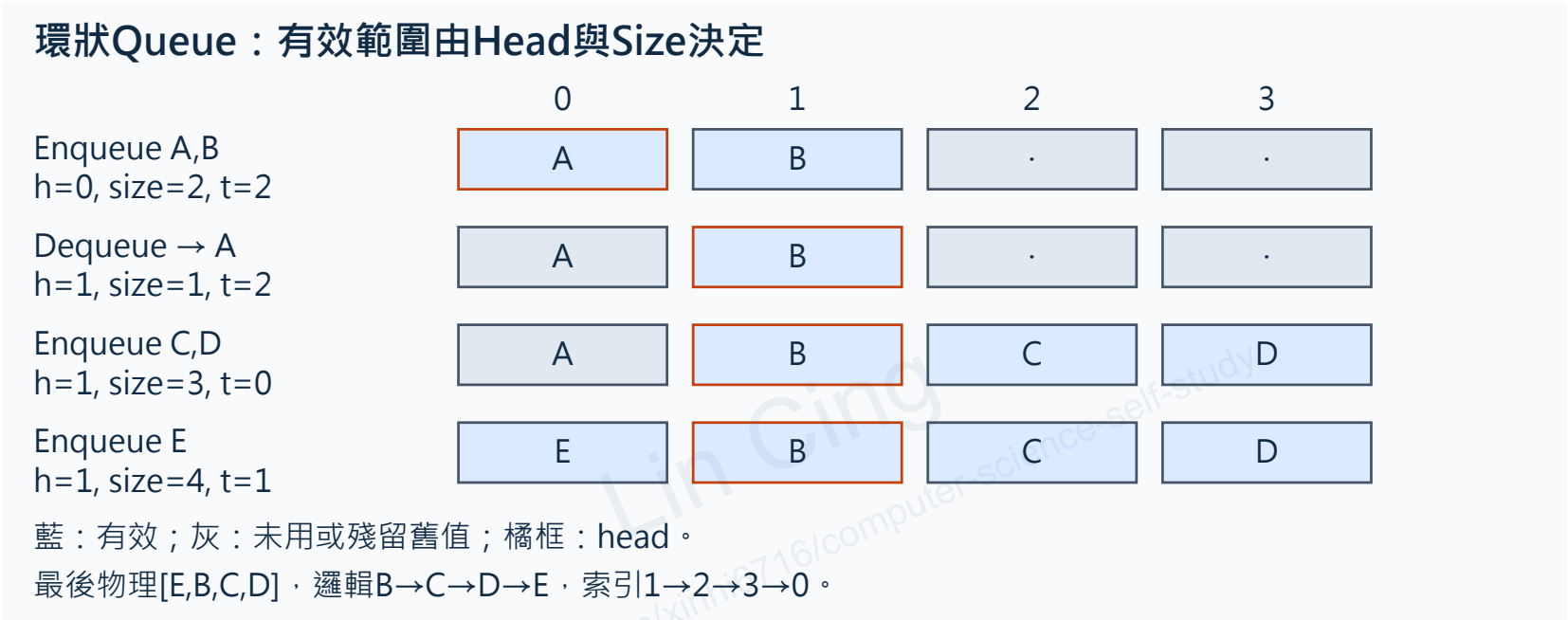
用Q[0..m−1]、head、size，tail定義為下一個寫入位置：tail=(head+size) mod m。第j個邏輯元素位於Q[(head+j) mod m]，0≤j<size。

```
enqueue(x): if size==m: error overflow
            Q[(head+size) mod m]=x; size++
dequeue():  if size==0: error underflow
            x=Q[head]; head=(head+1) mod m; size--; return x
```

Enqueue在原邏輯序列末尾新增，dequeue回傳原第0項並把head前進一格；模m處理跨尾端，故FIFO和索引不變量都維持。size=0與size=m均可有head=tail，必須靠size分辨。

原書也教「保留一格」版本：只存head/tail時，空為head=tail，滿為(tail+1) mod m=head，實際容量m−1。本章用size版本可容m筆，不能把兩種滿判斷混搭。

回繞不會改變FIFO順序



容量4：enqueue A,B；dequeue回A；enqueue C,D；再enqueue E。最終物理[E,B,C,D]，head=1、size=4、tail=1，邏輯B→C→D→E，索引1→2→3→0。此時再enqueue必上溢，下一次dequeue回B。

回繞不會改變FIFO順序 (續)

固定陣列的push/pop/enqueue/dequeue各最壞O(1)，前提是每筆資料可用O(1)搬移；建立m槽陣列本身需Θ(m)空間及相應配置。若滿時自動擴表，單次搬移可能Θ(m)，要改用第17章攤銷分析，不能沿用固定容量的單次最壞界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

10.2 Linked lists

陣列由索引決定鄰接，linked list由next連結決定邏輯順序；位置相鄰不要求記憶體地址相鄰。Singly linked只有next，doubly linked另有prev。無序串列search最壞必看n筆，Θ(n)；已持有合法節點handle的局部splice則可O(1)。

沒有sentinel時，首節點prev=NIL、尾節點next=NIL，刪除要另改head/tail。Circular doubly linked的sentinel N是實際的dummy節點：N.next指首、N.prev指尾，兩端接回N；空串列N.next=N.prev=N。

表示不變量：沿next由N回到N恰訪問每個資料節點一次，且每條邊u.next=v都有v.prev=u。N不代表任何資料鍵，search遇N表示失敗，delete不可刪N。

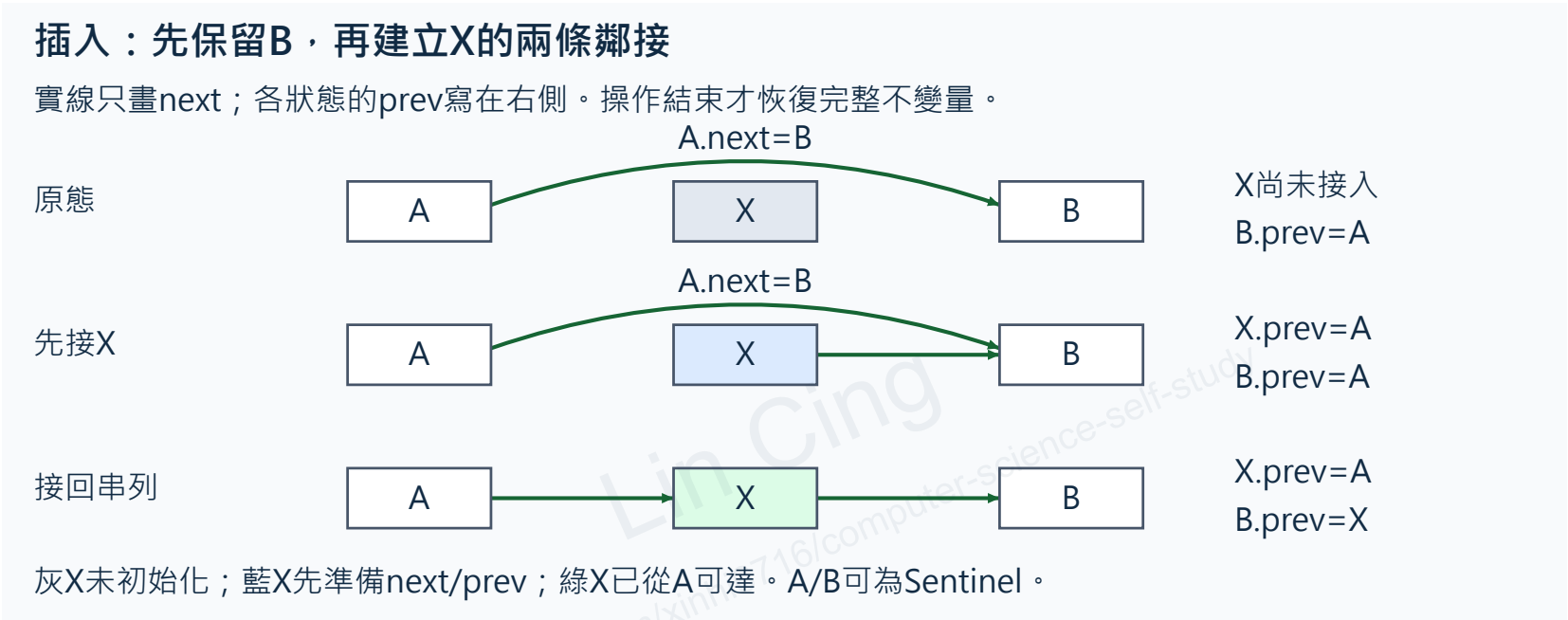
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先保存舊鄰居，再接入新節點

要在已相鄰的A、B間插入未在任何串列的新X，先保存B=A.next，依序寫X.prev=A、X.next=B，最後寫A.next=X、B.prev=X。讀舊鄰居必須在覆寫其來源前完成。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先保存舊鄰居，再接入新節點 （續）



中間步驟不必每一步都滿足完整雙向不變量；這是單執行緒、整個操作結束後恢復不變量的規格。若有並行讀者或中途例外，需另設同步/交易規則，不能把這四次寫入當成原子操作。

終態A.next=X、X.prev=A、X.next=B、B.prev=X，其餘邊不變，所以全域循環與雙向一致性維持。A或B可為sentinel，同一段程式涵蓋首插、尾插及空串列插入（A=B=N）。

刪除的前提、邊界與成本

對有效資料節點x，先保存a=x.prev、b=x.next，寫a.next=b、b.prev=a。原本a→x→b改成a→b，反向連結同步，其他邊不變。只有一個資料節點時a=b=N，兩次寫入恢復N自環。

例N↔A↔B↔N刪A後是N↔B↔N，仍有B，不能稱為空串列。可把x.next/x.prev清成NIL作除錯，但清除不會使外部舊handle自動失效，也不能取代正確的生命週期規則。

O(1)刪除的前提是呼叫者已給「目前屬於此串列、尚未刪除」的有效handle；若只有鍵，還要先search，最壞Θ(n)。若每次都從頭掃描驗證membership，也會花Θ(n)，不能省略這項成本後仍宣稱整次O(1)。

Singly linked已知前驅可O(1)刪其後繼；只有要刪的x通常不夠，特別是尾節點無法靠複製後繼解決。複製後繼還會改變identity，不是一般handle語義下的任意節點刪除。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

10.3 Implementing pointers and objects

沒有原生pointer也能用slot索引當handle。容量m的平行陣列key[0..m−1]、next、prev以同一索引i表示一個物件，NIL採−1，避免與合法slot0混淆。

例active list為slot0→slot2，key[0]=10、key[2]=30；next[0]=2、prev[2]=0。Logical successor是next[0]=2，沒有理由是物理相鄰slot1。

若改用單一陣列，每個固定三欄物件的base=3i，key=A[base]、next=A[base+1]、prev=A[base+2]。必須明定儲存的pointer是slot索引i還是base：本例欄內仍存slot索引，讀下個物件時重新乘3。原書也可直接存base位址，則不可再乘3。

多陣列與單陣列只是layout不同，不改變串列語義。若物件可有不同長度，簡單3i映射就不適用，需要額外的區塊配置/長度資訊，不能直接套以下固定slot free list。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Free list是未配置Slot的Stack

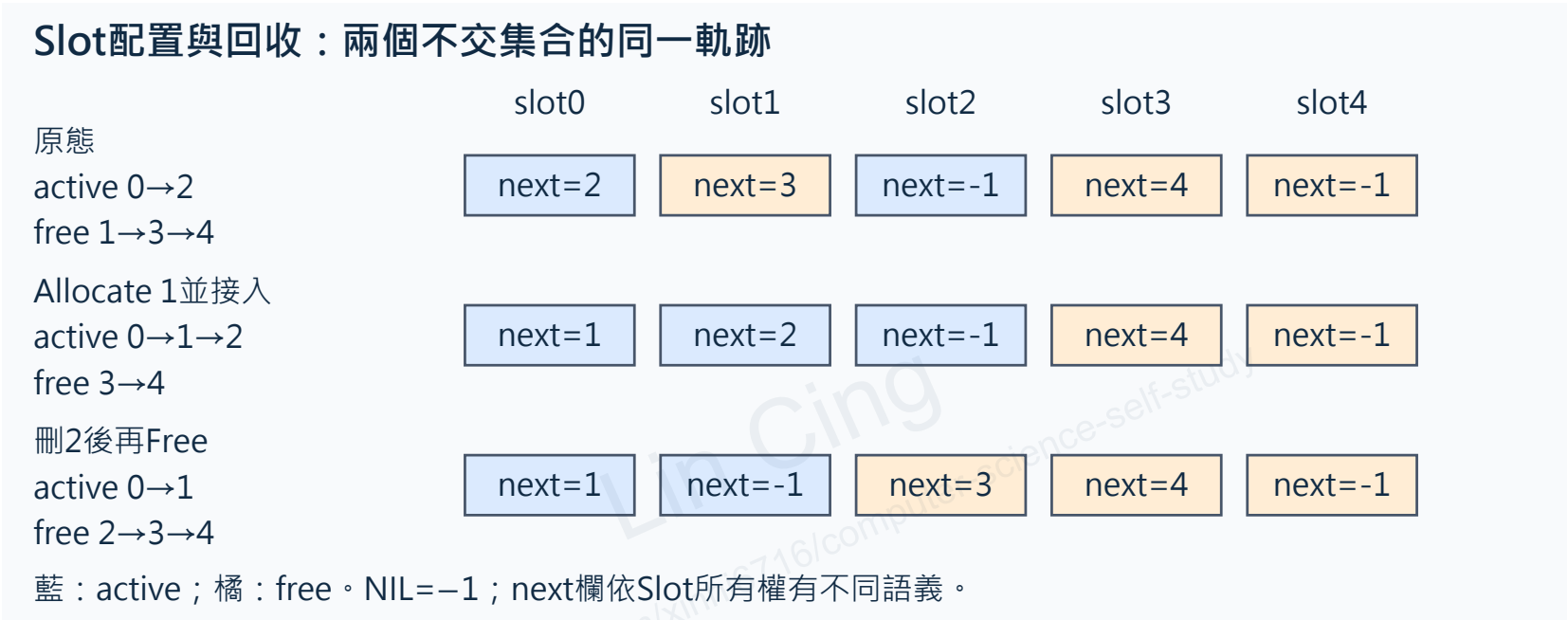
使用free指向第一個未配置slot，空時NIL；free slot的next欄重新作為free list連結。Allocate取首slot，free把已解除連結的slot插回首端：

```
allocate(): if free==NIL: error out-of-space
            x=free; free=next[x]; return x
free-object(x): next[x]=free; free=x
```

所有slot分成已配置與free兩個不交集合，聯集為全部m個slot；free list無環、不重複。Allocate把一個slot由free轉為已配置，呼叫者再初始化key/prev/next並接入資料結構。Free前必須先把x從所有持有它的結構移除，且x尚未free，否則可能破壞active邊或形成free list自環。

Free list只讀next，不需要把free slot的prev/key清零；重新配置後，必須在讀取前初始化要用的欄位。此配置器本身O(1)，不包含search、解構大型satellite data或主動清除所有外部引用。

配置、接入、解除、回收的同一軌跡



初態active0→2、free1→3→4。Allocate回1，free改3；初始化slot1並接於0與2之間，active0→1→2。接著先從active解除2，再free-object(2)，得active0→1、free2→3→4。

配置、接入、解除、回收的同一軌跡 （續）

若free(2)先改next[2]，卻還讓active節點指向2，下一次走訪就會跑入free slots。若同一slot再配置給另一物件，舊handle=2也不會自動分辨「舊物件2」與「新物件2」；使用者不可在free後繼續使用舊handle。若系統必須偵測這種stale handle，可另用(slot,generation)並驗證版本，但那是增加的表示規格。

把物件搬到緊密位置可能改善locality，卻會改變索引handle。只有能更新全部參照、或透過穩定間接表定位時才可搬移；「原物件內容相同」不代表所有指向舊位置的連結仍正確。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

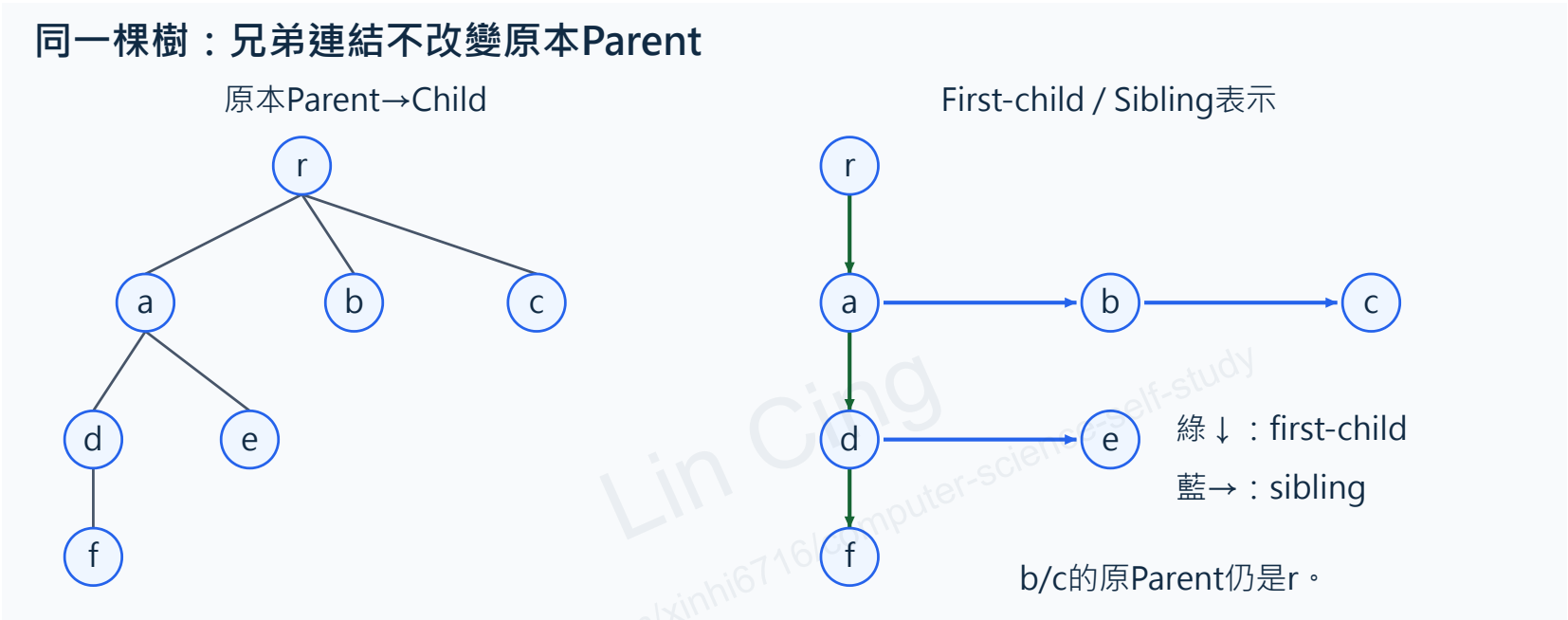
10.4 Representing rooted trees

Binary tree節點可存parent、left、right，根parent=NIL、缺child用NIL；空樹root=NIL。一般有根樹仍是每個非根節點恰一個parent，差別在child數可不固定，不是可以多個parent。

First-child/right-sibling（最左孩子/右兄弟）每節點存first、sibling與可選parent。first[x]是x的第一個child；sibling[x]是與x同父的下一個child。最後一個兄弟sibling=NIL，無child者first=NIL，root沒有兄弟。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

10.4 Representing rooted trees （續）



同一例r有a,b,c，a有d,e，d有f。first[r]=a；sib[a]=b、sib[b]=c；first[a]=d、sib[d]=e；first[d]=f。sib邊不是原樹的親子邊，例如b的原parent仍是r，不是a。

從表示法重建走訪與操作成本

```
visit-tree(x):
    if x==NIL: return
    output key[x]
    c=first[x]
    while c!=NIL:
        visit-tree(c)
        c=sibling[c]
```

每個node輸出一次；每個原樹parent的while恰列出其child一次，全樹child總數n−1，所以時間Θ(n)，不是把n個node各乘一個最壞n。Stack深度取決於原樹高度h，O(h+1)，鏈狀可Θ(n)，不是無條件O(log n)。此例preorder為r,a,d,f,e,b,c。

列x的全部children需Θ(deg(x)+1)，parent欄存在時回parent O(1)；只存parent則找全部children可能需掃全部node。表示法要配合操作：heap用完整樹陣列索引省pointer，disjoint-set只往根走可只存parent。

若把first/sibling機械地當binary left/right遞迴，起點若有兄弟也會走進它們；那是「以x開頭的兄弟森林」，不一定是原樹以x為根的子樹。上面先列children的版本會停在正確的原子樹邊界。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 10。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study